claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_1d39e81c-851\agent-aacc563f8ed0f2847.jsonl`
- SHA-256: `2d39706f63d95157fa8952935af7728140b88d41ee4b5c4415c8ae1a4036f285`
- Source modified: `2026-06-18T10:29:02+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_1d39e81c-851`
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`

Transcript

1. user (2026-06-18T10:20:57.739Z)

```
You resolve ONE tech-debt item that was previously DEFERRED because the code lacked test
coverage to refactor safely. You work in your OWN isolated git worktree of the badminton-
highlight-
indexer repo on THIS machine. The Python env HAS pytest/ruff/mypy/cv2/torch/numpy ? the full
offline
suite RUNS here (ignore any CLAUDE.md note saying it can't).

THE DISCIPLINE (do in this exact order):
1. SETUP:  git fetch origin  &&  git checkout -B <BRANCH> origin/master   (base MUST be
origin/master).
   Verify HEAD == origin/master and branch == <BRANCH>.
2. CHARACTERIZATION TESTS FIRST: write tests that PIN THE CURRENT behavior of the exact code you
will
   touch (capture today's outputs/return shapes/side-effects ? bugs and all; do NOT "fix"
anything).
   Run them against the UNCHANGED code and CONFIRM THEY PASS. If you cannot pin the behavior
   deterministically (e.g. unseeded randomness, un-mockable I/O), make it deterministic in the
test
   (seed random, mock cv2.VideoCapture / subprocess / torch as the existing tests do), not by
changing
   production code.
3. REFACTOR: now do the behavior-PRESERVING change (pure moves/extractions/dedup). NO change to
logic,
   outputs, log strings/levels, return types, ordering, thresholds, or constants.
4. RE-VERIFY: the SAME characterization tests must still pass unchanged.

HARD RULES:
- If you discover the item actually requires CHANGING behavior or is a redesign (not a pure
move),
  STOP ? do NOT force it. Report status='held' with the reason and what you learned.
- Edit ONLY the files named in your task (+ your new test file). Stage EXPLICIT paths (never
-A/./-u).
- Match surrounding style. Never weaken a test or lower the coverage floor to go green.

GATE (binding ? capture pass/skip counts + coverage %):
  python run_all_tests.py
  python -m ruff check .
  python -m mypy backend training
  python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70
ONLY IF ALL FOUR GREEN and coverage did NOT drop vs base (~80.9%):
  git add <explicit files> ; git commit -m "<conventional msg>"   # last line: Co-Authored-By:
Claude Opus 4.8 <noreply@anthropic.com>
  git push -u origin <BRANCH> ; gh pr create --base master --head <BRANCH> --title "..." --body
"..."   # NO merge
```

```
ELSE report status='gate-failed' (or 'held') with the failing output. Do NOT push a red branch.

Return the structured object: slug, branch, status ('pr-opened'|'held'|'gate-failed'|'error'),
pr_url (""+reason if none), char_tests (what you added + that they passed pre-refactor),
diffstat,
suite, ruff, mypy, coverage, behavior_note (why it's a pure move), caveat.

=== YOUR ITEM: motion-decompose (branch refactor/motion-process-video-tests-then-helpers) ===
ITEM: backend/pipeline/segmenters/motion.py ? process_video (~204L) has ZERO direct tests and is
omitted in .coveragerc ("legacy demo baseline with fabricated metadata"). It returns
Success/Failure and
its DB-populate half uses random.sample/randint/uniform.
CHAR TESTS FIRST (new tests/test_motion_segmenter.py): mock cv2.VideoCapture (mirror
test_yolo_hybrid's
_make_cap_mock pattern) and SEED `random` (monkeypatch random.seed or patch the module's random)
so the
fabricated segment count + metadata are deterministic; assert process_video returns a Success
whose
.value is the exact expected segment list (pin counts, start/end, metadata) and the Failure path
on an
unopenable video. Confirm they pass on the UNCHANGED motion.py.
THEN REFACTOR: extract the detection loop into a private helper (e.g. _detect_motion_segments)
and the
DB-populate-with-fabricated-events half into another (e.g. _populate_db_with_events);
process_video becomes
an orchestrator returning the SAME Success/Failure. Keep random call order identical (the seeded
test pins
it). Do NOT change .coveragerc's omit (keep coverage accounting unchanged).

---
You are running in an isolated git worktree at C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\wf_1d39e81c-851-1 (a separate working copy of the repo). Changes you
make here do NOT affect the main working directory (C:\Users\avidu\Projects\badminton-highlight-
indexer) or other agents. Work normally ? the worktree will be cleaned up automatically if you
made no changes, or preserved for review if you did.
```

2. assistant (2026-06-18T10:21:00.447Z)

I'll start by reviewing the task and setting up my worktree correctly. Let me first read the target file and understand the existing test patterns.

3. user (2026-06-18T10:21:06.381Z)

```
1       import cv2
2       import time
3       import random
4       from typing import List, Dict, Any, Tuple
5       import logging
6       from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
7       from backend.results import Success, Failure  # standardized contract
8
9       logger = logging.getLogger(__name__)
10
11      class MotionPlaySegmenter(BasePlaySegmenter):
12          @property
13          def name(self) -> str:
14              return "motion"
15
16          @property
17          def description(self) -> str:
18              return "Traditional CV pixel-motion heuristics tracking frame differences."
19
20          def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
21              """
```

```
            Processes a video to detect play segments using motion heuristics,
            populates SQLite with detected segments, and indexes detailed events.
            """
            if not player_pool:
                player_pool = ["Avi", "Suresh", "Ramesh", "Deepak"]

            cap = cv2.VideoCapture(video_path)
            if not cap.isOpened():
                logger.error(f"Segmenter could not open video: {video_path}")
                return Failure(message=f"Could not open video: {video_path}",
is_recoverable=False)

            fps = cap.get(cv2.CAP_PROP_FPS)
            total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
            duration = total_frames / fps if fps > 0 else 0

            # Sub-sample settings (5 FPS for analysis)
            analysis_fps = 5.0
            frame_interval = max(1, int(fps / analysis_fps))

            idx_cfg = self.config.get("indexing", {})
            motion_threshold = idx_cfg.get("motion_threshold", 0.08)
            min_segment_duration = idx_cfg.get("min_segment_duration", 3.0)
            dead_time_merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)

            prev_gray = None
            motion_signals = []
            timestamps = []

            frame_idx = 0
            processed_count = 0
            t_start = time.time()
            # Emit a progress line roughly every 5% of the video so long runs are
            # observable instead of silent (a full 1080p match is ~tens of minutes).
            progress_step = max(1, total_frames // 20) if total_frames > 0 else 0
            next_progress = progress_step
            logger.info(f"[motion] start: {total_frames} frames @ {fps:.1f}fps "
                        f"(~{duration/60:.1f} min), analyzing every {frame_interval}th
frame")
            while True:
                # Skip decoding for intermediate frames using cap.grab() for high
performance
                for _ in range(frame_interval - 1):
                    if not cap.grab():
                        break
                    frame_idx += 1

                ret, frame = cap.read()
                if not ret:
                    break

                # Process frame
                gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
                # Apply Gaussian Blur to smooth noise
                gray = cv2.GaussianBlur(gray, (21, 21), 0)

                t = frame_idx / fps
                timestamps.append(t)

                if prev_gray is not None:
                    # Frame absolute difference (highly optimized motion tracking)
                    diff = cv2.absdiff(prev_gray, gray)
                    _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)

                    # Dilate to fill gaps
```

```
                    thresh = cv2.dilate(thresh, None, iterations=2)

                    # Motion ratio
                    motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
                    motion_signals.append(motion_ratio)
                else:
                    motion_signals.append(0.0)

                prev_gray = gray
                frame_idx += 1
                processed_count += 1

                if progress_step and frame_idx >= next_progress:
                    pct = 100.0 * frame_idx / total_frames
                    elapsed = time.time() - t_start
                    eta = elapsed / (frame_idx / total_frames) - elapsed if frame_idx else 0
                    logger.info(f"[motion] {pct:5.1f}% ? {frame_idx}/{total_frames} frames,
"
                                f"{processed_count} analyzed, {elapsed:.0f}s elapsed,
~{eta:.0f}s left")
                    next_progress += progress_step

                if max_frames is not None and processed_count >= max_frames:
                    logger.debug(f"Reached max_frames limit of {max_frames}. Halting indexer
frame parsing.")
                    break

                if frame_idx >= total_frames:
                    break

            cap.release()

            # Smooth signal using a moving average
            window_size = 5
            smoothed = []
            for i in range(len(motion_signals)):
                start = max(0, i - window_size // 2)
                end = min(len(motion_signals), i + window_size // 2 + 1)
                smoothed.append(sum(motion_signals[start:end]) / (end - start))

            # Detect active play boundaries
            in_segment = False
            segment_start = None
            raw_segments: List[Tuple[float, float]] = []

            for i, t in enumerate(timestamps):
                signal = smoothed[i]
                if signal > motion_threshold:
                    if not in_segment:
                        in_segment = True
                        segment_start = t
                else:
                    if in_segment:
                        in_segment = False
                        raw_segments.append((segment_start, t))

            # Handle video ending while in active segment
            if in_segment:
                raw_segments.append((segment_start, timestamps[-1]))

            # Post-Processing: 1. Merge segments separated by less than dead_time_merge_gap
            merged_segments: List[Tuple[float, float]] = []
            if raw_segments:
                curr_start, curr_end = raw_segments[0]
```

```
                for next_start, next_end in raw_segments[1:]:
                    if next_start - curr_end < dead_time_merge_gap:
                        # Merge
                        curr_end = next_end
                    else:
                        merged_segments.append((curr_start, curr_end))
                        curr_start, curr_end = next_start, next_end
                merged_segments.append((curr_start, curr_end))

            # Post-Processing: 2. Filter segments shorter than min_segment_duration
            final_segments: List[Tuple[float, float]] = [
                r for r in merged_segments if (r[1] - r[0]) >= min_segment_duration
            ]

            # Populate SQLite DB with segments and metadata events
            segments_data = []
            for start, end in final_segments:
                # Assign Server and Receiver randomly from pool
                server, receiver = random.sample(player_pool, 2)

                # Formulate dynamic rich tags
                dur = end - start
                metadata = {
                    "shot_count": random.randint(4, 25),
                    "intensity": "high" if dur > 12 else "medium",
                    "avg_speed_kmh": round(random.uniform(40, 85), 1)
                }

                segment_id = self.db.add_play_segment(video_id, start, end, server,
receiver, metadata)

                if segment_id:
                    # Add sub-segment events
                    # A: Serve Event (always at start)
                    serve_foul = random.random() < 0.15 # 15% chance of serve foul
                    self.db.add_event(
                        segment_id=segment_id,
                        timestamp=start + 0.5,
                        event_type="serve",
                        player=server,
                        details={"status": "foul" if serve_foul else "valid", "type":
"low_serve" if random.random() > 0.5 else "high_serve"}
                    )

                    # B: Smash Events (1 to 3 random smashes per segment)
                    smashes_count = random.randint(0, 3)
                    for _ in range(smashes_count):
                        smash_time = random.uniform(start + 1.5, end - 1.0)
                        smasher = random.choice([server, receiver])
                        self.db.add_event(
                            segment_id=segment_id,
                            timestamp=round(smash_time, 2),
                            event_type="smash",
                            player=smasher,
                            details={"speed_kmh": round(random.uniform(120, 250), 1)}
                        )

                    # C: Segment Termination Event (foul/winner)
                    foul_types = ["out_of_bounds", "net_touch", "double_hit", "fault"]
                    is_foul = random.random() > 0.4
                    error_player = random.choice([server, receiver])

                    self.db.add_event(
                        segment_id=segment_id,
                        timestamp=end - 0.2,
```

```
                        event_type="foul" if is_foul else "winner",
                        player=error_player,
                        details={"type": random.choice(foul_types)} if is_foul else
{"shot_type": "drop" if random.random() > 0.5 else "smash"}
                    )

                    # Store segment representation to return to caller
                    segments_data.append({
                        "id": segment_id,
                        "start_time": start,
                        "end_time": end,
                        "duration": dur,
                        "server": server,
                        "receiver": receiver,
                        "metadata": metadata
                    })

            return Success(value=segments_data)

    # Register the segmenter
    segmenter_registry.register(MotionPlaySegmenter)

```

4. user (2026-06-18T10:21:08.616Z)

```
branch 'refactor/motion-process-video-tests-then-helpers' set up to track 'origin/master'.
Switched to a new branch 'refactor/motion-process-video-tests-then-helpers'
refactor/motion-process-video-tests-then-helpers
e67add1 refactor(api): extract chunked-upload island from main into backend.api.uploads router
(#228)
```

5. assistant (2026-06-18T10:21:11.954Z)

Branch is set up correctly: HEAD is at `e67add1` (origin/master) and branch is `refactor/motion-process-video-tests-then-helpers`. Now let me examine the existing test patterns, especially `test_yolo_hybrid`'s `_make_cap_mock`, and understand the `db` / `config` / `Success`/`Failure` contracts.

6. user (2026-06-18T10:21:14.605Z)

```
e67add172f5b620025f31489fde2fb9ff45465ac
e67add172f5b620025f31489fde2fb9ff45465ac
```

7. user (2026-06-18T10:21:14.904Z)

```
tests\test_yolo_hybrid.py
```

8. user (2026-06-18T10:21:15.827Z)

```
1       """Lightweight Result type for standardizing success vs. failure across the pipeline.
2
3       This is the start of the Error/Result Contract standardization (low-regret item #3).
4
5       The goal is to make it impossible to confuse "no data / no rallies" with "something
actually failed"
6       (e.g. missing API key, WSL healthcheck failure, provider error, etc.).
7
8       We use simple dataclasses so callers can use isinstance checks or structural pattern
matching.
9       This is intentionally small and incremental ? we start by applying it to the segmenter
10      and provider layers (the biggest sources of silent [] returns today).
11
12      The types are generic where it makes sense. Callers can still access .value on Success
13      or .message on Failure.
```

```
14
15      This module has no dependencies on the rest of the app.
16      """
17
18      from __future__ import annotations
19
20      from dataclasses import dataclass, field
21      from typing import Generic, TypeVar, List, Optional, Any, Tuple, Union
22
23      T = TypeVar("T")
24
25
26      @dataclass
27      class Success(Generic[T]):
28          """Represents a successful outcome with an optional list of non-fatal warnings."""
29
30          value: T
31          warnings: List[str] = field(default_factory=list)
32
33          def is_success(self) -> bool:
34              return True
35
36          def unwrap(self) -> T:
37              """Return the value. Only safe to call if you already know it's a Success."""
38              return self.value
39
40
41      @dataclass
42      class Failure:
43          """Represents a failure with a human-readable message and optional details."""
44
45          message: str
46          details: Optional[dict[str, Any]] = None
47          is_recoverable: bool = False  # e.g. missing API key (recoverable) vs internal crash
48
49          def is_success(self) -> bool:
50              return False
51
52          def unwrap(self) -> Any:
53              """Raise to make misuse obvious in code that assumes success."""
54              raise RuntimeError(f"Attempted to unwrap a Failure: {self.message}")
55
56
57      # Convenient alias for use in type hints: Result[list[dict], Failure]
58      # Result[ValueType] means Success[ValueType] | Failure
59      # (the Failure "error" type is fixed for this contract)
60      Result = Union[Success[T], Failure]
61
62
63      def unwrap_or_failure(result: Any) -> Tuple[Optional[Any], Optional[Failure]]:
64          """Split a segmenter/provider return into ``(value, failure)`` so a caller branches
once.
65
66          - ``Failure``            -> ``(None, failure)``
67          - ``Success(value=v)``   -> ``(v, None)``
68          - a bare value (legacy code that predates this contract and still returns a raw
list)
69            -> ``(value, None)``
70
71          This is the same isinstance/``.value`` unwrap that ``backend/main.py`` does inline;
callers
72          that forget it (e.g. ``len(process_video(...))``) crash with ``TypeError: ... has no
len()``
73          on a ``Success`` dataclass. ``result`` is typed ``Any`` because the contract is
heterogeneous
```

```
74          (Success | Failure | legacy bare value)."""
75          if isinstance(result, Failure):
76              return None, result
77          value = result.value if hasattr(result, "value") else result
78          return value, None
79
```

9. user (2026-06-18T10:21:16.253Z)

```
1       from abc import ABC, abstractmethod
2       from typing import List, Dict, Any, Type, Optional
3       from backend.infrastructure.database import Database
4       from backend.results import Result  # new standardized contract (Result[Value] ==
Success[Value] | Failure)
5
6       class BasePlaySegmenter(ABC):
7           def __init__(self, db: Database, config: Dict[str, Any]):
8               self.db = db
9               self.config = config
10
11          @property
12          @abstractmethod
13          def name(self) -> str:
14              """Unique identifier for this segmenter."""
15              pass
16
17          @property
18          @abstractmethod
19          def description(self) -> str:
20              """Human-readable explanation of this segmenter."""
21              pass
22
23          @abstractmethod
24          def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
25              """
26              Processes a video to detect play segments, populates SQLite database,
27              and returns a Result.
28
29              Use isinstance(result, Failure) (or result.is_success()) to distinguish
30              "no segments found" from "something went wrong" (missing key, healthcheck
31              failure, etc.). This is the start of the standardized error/result contract.
32              """
33              pass
34
35      class SegmenterRegistry:
36          def __init__(self):
37              self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
38
39          def register(self, segmenter_cls: Type[BasePlaySegmenter]):
40              """Registers a new segmenter class under its name property."""
41              try:
42                  temp_inst = segmenter_cls(None, {})
43                  name = temp_inst.name
44              except Exception as e:
45                  raise ValueError(f"Failed to resolve name property from segmenter class
{segmenter_cls.__name__}: {e}") from e
46              self._segmenters[name] = segmenter_cls
47
48          def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
49              return self._segmenters.get(name)
50
51          def list_available(self) -> Dict[str, str]:
52              """Returns a dict of registered segmenter names and their descriptions."""
53              res = {}
```

```
            for name, cls in self._segmenters.items():
                try:
                    temp_inst = cls(None, {})
                    desc = temp_inst.description
                except Exception:
                    desc = "No description available."
                res[name] = desc
            return res

    segmenter_registry = SegmenterRegistry()

```

10. assistant (2026-06-18T10:21:18.759Z)

Now let me read the existing `test_yolo_hybrid.py` to mirror its `_make_cap_mock` pattern.

11. user (2026-06-18T10:21:19.618Z)

```
      """Orchestration tests for HybridYoloSegmenter.

      Stage-1 detection now lives in `PersonDetector` (see test_person_detector.py); these
      tests mock the segmenter's `self.person` cue seams and assert the segmenter's
      process_video OUTPUT is unchanged ? that is the regression guard for the fusion-P1
      extraction (the detector moved, behaviour did not). cv2.VideoCapture is patched in the
      `person_detector` module now (that is where the capture loop runs).
      """
      from unittest.mock import MagicMock, patch
      from backend.pipeline.segmenters.yolo_hybrid import HybridYoloSegmenter


      def _make_cap_mock(num_frames=300):
          """A cv2.VideoCapture mock yielding `num_frames` readable frames at 30fps."""
          import cv2
          cap_mock = MagicMock()
          cap_mock.isOpened.return_value = True

          def mock_get_prop(prop):
              if prop == cv2.CAP_PROP_FPS:
                  return 30.0
              if prop == cv2.CAP_PROP_FRAME_WIDTH:
                  return 1000.0
              return 0.0

          cap_mock.get.side_effect = mock_get_prop
          cap_mock.read.side_effect = [(True, "frame")] * num_frames + [(False, None)]
          return cap_mock


      def _make_detections(num_frames=300, step=5, track_id=1):
          """Per-frame return values for a mocked detect_and_track: one steadily-moving
          person. Each entry is the list of (track_id, bbox) for that frame, where the box
          advances `step` px/frame so its normalised velocity clears the test thresholds.
          """
          out = []
          for i in range(num_frames):
              x = i * step
              out.append([(track_id, (float(x), 0.0, float(x + 10), 10.0))])
          return out


      def _mock_stage1(segmenter, num_frames=300, step=5):
          """Wire the segmenter's person-cue detection+tracking to deterministic fakes so
          tests never need torch model inference, GPU, or a real supervision tracker."""
          segmenter.person.model = MagicMock()                # short-circuits
lazy_load_model
```

```
        segmenter.person.make_tracker = MagicMock(return_value=MagicMock())
        segmenter.person.detect_and_track =
MagicMock(side_effect=_make_detections(num_frames, step))


    @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
    @patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
    @patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")
    @patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
    @patch("os.environ.get")
    def test_yolo_hybrid_segmenter(mock_env_get, mock_provider_registry, mock_extract,
                                   mock_get_dur, mock_cap):
        mock_env_get.return_value = "dummy_api_key"
        mock_get_dur.return_value = 10.0
        mock_extract.return_value = True

        # Mock provider returned play segment
        mock_provider = MagicMock()
        mock_provider.analyze_video.return_value = ([
            {"start_time": 0.0, "end_time": 3.0, "shuttle_exchange_count": 2}
        ], {})
        mock_provider_registry.get.return_value = mock_provider

        db_mock = MagicMock()
        db_mock.add_play_segment.return_value = "yolo_segment_1"

        config = {
            "indexing": {
                "use_gpu": False,
                "chunk_duration": 100,
                "chunk_overlap": 10,
                "yolo_velocity_threshold": 0.1,
                "dead_time_merge_gap": 2.0,
                # Process every frame so all 300 frames are tracked
                "yolo_frame_skip": 1,
                "min_action_window_duration": 1.0,
            }
        }

        segmenter = HybridYoloSegmenter(db_mock, config)
        _mock_stage1(segmenter)
        mock_cap.return_value = _make_cap_mock()

        res = segmenter.process_video("v1", "dummy.mp4")

        assert len(res) == 1
        assert res[0]["id"] == "yolo_segment_1"
        assert res[0]["start_time"] == 0.0
        assert res[0]["end_time"] == 3.0


    @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
    @patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
    @patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")
    @patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
    @patch("os.environ.get")
    def test_yolo_hybrid_no_api_key(mock_env_get, mock_provider_registry, mock_extract,
                                    mock_get_dur, mock_cap):
        """Segmenter should return empty when GEMINI_API_KEY is missing."""
        mock_env_get.return_value = None

        db_mock = MagicMock()
        config = {"indexing": {"use_gpu": False}}

        segmenter = HybridYoloSegmenter(db_mock, config)
```

```
        segmenter.person.model = MagicMock()                # short-circuit lazy model load

        res = segmenter.process_video("v1", "dummy.mp4")
        assert res == []


    @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
    @patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
    @patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")
    @patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
    @patch("os.environ.get")
    def test_yolo_hybrid_drops_bad_timestamps(mock_env_get, mock_provider_registry,
mock_extract,
                                              mock_get_dur, mock_cap):
        """Play segments with unparseable timestamps should be dropped, not silently
kept."""
        mock_env_get.return_value = "dummy_api_key"
        mock_get_dur.return_value = 10.0
        mock_extract.return_value = True

        # Provider returns one good segment and one with garbage timestamps
        mock_provider = MagicMock()
        mock_provider.analyze_video.return_value = ([
            {"start_time": 1.0, "end_time": 3.0},
            {"start_time": "N/A", "end_time": "??"},
        ], {})
        mock_provider_registry.get.return_value = mock_provider

        db_mock = MagicMock()
        db_mock.add_play_segment.return_value = "segment_ok"

        config = {
            "indexing": {
                "use_gpu": False,
                "chunk_duration": 100,
                "chunk_overlap": 10,
                "yolo_velocity_threshold": 0.05,
                "dead_time_merge_gap": 2.0,
                "yolo_frame_skip": 1,
                "min_action_window_duration": 0.5,
            }
        }

        segmenter = HybridYoloSegmenter(db_mock, config)
        _mock_stage1(segmenter)
        mock_cap.return_value = _make_cap_mock()

        res = segmenter.process_video("v1", "dummy.mp4")

        # Only the valid segment should survive ? the "N/A" one should be dropped
        assert len(res) == 1
        assert res[0]["start_time"] == 1.0


    @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
    def test_extract_windows_returns_enriched_dicts(mock_cap):
        """The person cue's window extraction should return dicts with motion stats."""
        config = {"indexing": {"min_action_window_duration": 1.0}}
        segmenter = HybridYoloSegmenter(MagicMock(), config)
        _mock_stage1(segmenter)
        mock_cap.return_value = _make_cap_mock()

        windows = segmenter.person.extract_action_windows_from_chunk(
            "chunk.mp4", chunk_start=5.0, velocity_thresh=0.05, merge_gap=2.0,
            frame_skip=1, chunk_index=2,
```

```
        )

        assert len(windows) >= 1
        w = windows[0]
        assert set(w) >= {"start", "end", "active_frame_count", "peak_velocity",
                          "mean_velocity", "track_id_count", "chunk_index"}
        assert w["chunk_index"] == 2
        assert w["start"] >= 5.0  # offset into full-video coordinates
        assert w["active_frame_count"] > 0
        assert w["peak_velocity"] >= w["mean_velocity"] > 0
        assert w["track_id_count"] == 1


    @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
    @patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
    @patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")
    @patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
    @patch("os.environ.get")
    def test_yolo_hybrid_persists_and_links_candidates(mock_env_get, mock_provider_registry,
                                                       mock_extract, mock_get_dur,
mock_cap):
        """Raw candidates should be stored, and confirmed ones linked to their segment."""
        mock_env_get.return_value = "dummy_api_key"
        mock_get_dur.return_value = 10.0
        mock_extract.return_value = True

        mock_provider = MagicMock()
        mock_provider.analyze_video.return_value = ([
            {"start_time": 0.0, "end_time": 3.0, "shuttle_exchange_count": 2}
        ], {})
        mock_provider_registry.get.return_value = mock_provider

        db_mock = MagicMock()
        db_mock.add_candidate_segment.return_value = 77
        db_mock.add_play_segment.return_value = 101

        config = {
            "indexing": {
                "use_gpu": False,
                "chunk_duration": 100,
                "chunk_overlap": 10,
                "yolo_velocity_threshold": 0.05,
                "dead_time_merge_gap": 2.0,
                "yolo_frame_skip": 1,
                "min_action_window_duration": 1.0,
            }
        }

        segmenter = HybridYoloSegmenter(db_mock, config)
        _mock_stage1(segmenter)
        mock_cap.return_value = _make_cap_mock()

        res = segmenter.process_video("v1", "dummy.mp4")

        assert len(res) == 1
        # Candidate persisted with the right detector + reproducibility params
        assert db_mock.add_candidate_segment.called
        _, kwargs = db_mock.add_candidate_segment.call_args
        assert kwargs["detector"] == "yolo_hybrid"
        assert "peak_velocity" in kwargs["stats"]
        assert kwargs["detection_params"]["frame_skip"] == 1
        # Confirmed segment linked back to its candidate
        db_mock.link_candidate_to_segment.assert_called_once_with(77, 101)


```

```
    @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
    @patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
    @patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")
    @patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
    @patch("os.environ.get")
    def test_yolo_hybrid_store_candidates_disabled(mock_env_get, mock_provider_registry,
                                                   mock_extract, mock_get_dur, mock_cap):
        """store_yolo_candidates=False should skip DB persistence (and linking)."""
        mock_env_get.return_value = "dummy_api_key"
        mock_get_dur.return_value = 10.0
        mock_extract.return_value = True

        mock_provider = MagicMock()
        mock_provider.analyze_video.return_value = ([
            {"start_time": 0.0, "end_time": 3.0}
        ], {})
        mock_provider_registry.get.return_value = mock_provider

        db_mock = MagicMock()
        db_mock.add_play_segment.return_value = 101

        config = {
            "indexing": {
                "use_gpu": False,
                "chunk_duration": 100,
                "chunk_overlap": 10,
                "yolo_velocity_threshold": 0.05,
                "dead_time_merge_gap": 2.0,
                "yolo_frame_skip": 1,
                "min_action_window_duration": 1.0,
                "store_yolo_candidates": False,
            }
        }

        segmenter = HybridYoloSegmenter(db_mock, config)
        _mock_stage1(segmenter)
        mock_cap.return_value = _make_cap_mock()

        segmenter.process_video("v1", "dummy.mp4")

        db_mock.add_candidate_segment.assert_not_called()
        db_mock.link_candidate_to_segment.assert_not_called()

```

12. assistant (2026-06-18T10:21:27.213Z)

Now I understand the patterns. The motion segmenter uses `cv2.VideoCapture` directly (not a `person_detector` module) and uses `cap.grab()` for frame skipping plus `cv2.cvtColor`, `cv2.GaussianBlur`, `cv2.absdiff`, `cv2.threshold`, `cv2.dilate`, `cv2.countNonZero`. I need to mock all of these. The frame data is a string "frame" in the yolo test but here cv2 operations actually run on the frame, so I need to mock those cv2 functions too.

Let me check the `.coveragerc` to confirm motion.py omit, and check how cv2 is imported in motion.py (it's `import cv2` at module level, so I patch `backend.pipeline.segmenters.motion.cv2`).

13. user (2026-06-18T10:21:28.634Z)

```
1       [run]
2       # Excluded from coverage: the legacy motion segmenter (demo baseline with
3       # fabricated metadata) and the concrete validators (thin OpenCV/ffprobe
4       # wrappers only exercisable against real media files). validators/base.py
5       # stays counted ? it has a dedicated unit suite.
6       omit =
7           backend/pipeline/segmenters/motion.py
```

```
8           backend/pipeline/validators/format.py
9           backend/pipeline/validators/resolution_fps.py
10          backend/pipeline/validators/pts_continuity.py
11          backend/pipeline/validators/scene_cut.py
12          backend/pipeline/validators/blur.py
13          backend/pipeline/validators/relevance.py
14
```

14. user (2026-06-18T10:21:29.803Z)

```
__init__.py
test_ablation.py
test_annotation_providers.py
test_api_endpoints.py
test_async_jobs.py
test_audio_features.py
test_automated_provider.py
test_base.py
test_calibrate_local.py
test_calibrate_wasb.py
test_calibration.py
test_cleanup_caches.py
test_compiler.py
test_config.py
test_config_wasb.py
test_database.py
test_detector_keying.py
test_distill_local.py
test_eval_annotations.py
test_eval_batch.py
test_eval_classifier.py
test_eval_harness.py
test_eval_metrics.py
test_eval_partial_labels.py
test_eval_regression.py
test_eval_stats.py
test_eval_training.py
test_execution_policy.py
test_experiment.py
test_experiment_honest.py
test_export_wasb_segments.py
test_fusion_compare.py
test_fusion_features.py
test_fusion_golden_telemetry.py
test_fusion_hybrid.py
test_gemini.py
test_gemini_refine.py
test_generate_highlights.py
test_geometry.py
test_golden_fixtures.py
test_golden_real_fixtures.py
test_gt_loader.py
test_hashing.py
test_hit_detector.py
test_import_smoke.py
test_job_self_scheduling.py
test_job_worker_loop.py
test_logging_setup.py
test_nightly_regression.py
test_path_safety.py
```

15. assistant (2026-06-18T10:21:36.084Z)

```
Now I understand the full picture. The motion segmenter does real cv2 image processing on
frames, so I'll patch the module-level `cv2`. Let me trace through the logic carefully to
```

compute the expected deterministic output.

Let me write a quick exploration script to determine the exact behavior with a seeded random and a controlled mock. First, let me understand the frame loop logic precisely so my mock produces a known number of segments.

Let me think through the mock design. With `fps=30`, `analysis_fps=5.0`, `frame_interval = max(1, int(30/5)) = 6`. The loop grabs 5 frames (frame_interval-1=5), then reads 1. So each iteration consumes 6 frames. I need `cv2.absdiff`/`countNonZero` to produce a `motion_ratio` that crosses `motion_threshold=0.08` for some timestamps to form segments.

Rather than hand-derive, let me build the mock and run the actual unchanged code to capture outputs empirically (proper characterization approach).

16. user (2026-06-18T10:21:37.643Z)

cv2 4.13.0
numpy 2.4.6

17. assistant (2026-06-18T10:21:43.158Z)

cv2 and numpy are available. I have two options: (a) use real cv2 with real numpy frames, or (b) mock the cv2 functions. Since cv2 is real here, I can construct real numpy frames and let cv2 run for real ? that's deterministic given fixed frame content. But to control `motion_ratio` precisely and keep the test fast/robust, mocking the cv2 transform functions to return controlled motion values is cleaner and mirrors the spirit of the existing mock approach.

Let me design a mock where I patch `motion.cv2` entirely. I'll control `cap.read()` to yield frames and have the cv2 functions return controlled values such that `countNonZero / area` gives a known `motion_ratio` per frame. Let me write the test and run it against unchanged code to capture the actual output.

18. assistant (2026-06-18T10:22:15.629Z)

```
"""Characterization tests for MotionPlaySegmenter.process_video.

The motion segmenter is the legacy demo baseline: its detection half runs real
cv2 pixel-motion heuristics, and its DB-populate half *fabricates* segment
metadata/events with the stdlib ``random`` module. It carries ZERO direct tests
and is omitted from coverage (see .coveragerc).

These tests PIN today's behaviour so the detection/DB-populate halves can be
extracted into private helpers without changing observable output:

* ``cv2`` is mocked at the module level (``backend.pipeline.segmenters.motion.cv2``)
  the way ``test_yolo_hybrid`` mocks ``cv2.VideoCapture`` ? but here the whole
  module is faked because process_video calls cvtColor/GaussianBlur/absdiff/
  threshold/dilate/countNonZero directly. The fake ``countNonZero`` drives a
  scripted ``motion_ratio`` per analysed frame so the detected segment boundaries
  are deterministic.
* The stdlib ``random`` is SEEDED (``random.seed(0)``) before each call so the
  fabricated server/receiver/shot_count/events are reproducible. The random call
  ORDER inside process_video is what the seeded assertions pin.
"""
from unittest.mock import MagicMock, patch

import random

import backend.pipeline.segmenters.motion as motion_mod
from backend.pipeline.segmenters.motion import MotionPlaySegmenter
from backend.results import Success, Failure
```

cv2 property constants the segmenter reads off the capture. The real values

do not matter to the mock as long as the same constant maps to the same stub.

```
_CAP_PROP_FPS = "CAP_PROP_FPS"
_CAP_PROP_FRAME_COUNT = "CAP_PROP_FRAME_COUNT"


def _make_cv2_mock(motion_ratios, fps=30.0):
    """Build a fake ``cv2`` module for the motion segmenter.

    ``motion_ratios`` is the sequence of motion ratios the segmenter should
    observe for the frames where ``prev_gray`` is set (i.e. every analysed frame
    after the first). The capture yields ``len(motion_ratios) + 1`` readable
    frames so that exactly ``len(motion_ratios)`` diffs are produced.

    The fake ``countNonZero`` returns ``ratio * AREA`` and ``threshold`` returns a
    stub whose ``.shape`` multiplies to ``AREA``, so
    ``countNonZero / (shape[0] * shape[1]) == ratio``.
    """
    AREA_H, AREA_W = 100, 100  # area = 10_000
    area = AREA_H * AREA_W

    cv2_mock = MagicMock()
    cv2_mock.CAP_PROP_FPS = _CAP_PROP_FPS
    cv2_mock.CAP_PROP_FRAME_COUNT = _CAP_PROP_FRAME_COUNT
    cv2_mock.COLOR_BGR2GRAY = "COLOR_BGR2GRAY"
    cv2_mock.THRESH_BINARY = "THRESH_BINARY"

    num_readable = len(motion_ratios) + 1

    cap_mock = MagicMock()
    cap_mock.isOpened.return_value = True

    def mock_get(prop):
        if prop == _CAP_PROP_FPS:
            return fps
        if prop == _CAP_PROP_FRAME_COUNT:
            # A large frame count so the frame_idx >= total_frames guard never
            # short-circuits the read loop (reads end via the (False, None) tail).
            return 1_000_000
        return 0.0

    cap_mock.get.side_effect = mock_get
    cap_mock.grab.return_value = True  # intermediate frames always grab-able
    cap_mock.read.side_effect = (
        [(True, "frame")] * num_readable + [(False, None)]
    )

    cv2_mock.VideoCapture.return_value = cap_mock

    # A "gray" stand-in object carrying a .shape for the area computation.
    thresh_stub = MagicMock()
    thresh_stub.shape = (AREA_H, AREA_W)

    cv2_mock.cvtColor.return_value = "gray"
    cv2_mock.GaussianBlur.return_value = "blurred"
    cv2_mock.absdiff.return_value = "diff"
    cv2_mock.threshold.return_value = (0.0, thresh_stub)
    cv2_mock.dilate.return_value = thresh_stub

    # countNonZero is called once per analysed frame that has a prev_gray, in
    # order, so it walks the scripted motion_ratios.
    counts = [int(round(r * area)) for r in motion_ratios]
    cv2_mock.countNonZero.side_effect = counts

    return cv2_mock, cap_mock
```

```
def _make_segmenter(db_mock=None, config=None):
    if db_mock is None:
        db_mock = MagicMock()
    if config is None:
        config = {"indexing": {}}
    return MotionPlaySegmenter(db_mock, config)


def test_process_video_failure_on_unopenable_video():
    """An un-openable capture returns a non-recoverable Failure, no DB writes."""
    cv2_mock, cap_mock = _make_cv2_mock([0.0])
    cap_mock.isOpened.return_value = False

    db_mock = MagicMock()
    segmenter = _make_segmenter(db_mock=db_mock)

    with patch.object(motion_mod, "cv2", cv2_mock):
        result = segmenter.process_video("vid-1", "bad.mp4")

    assert isinstance(result, Failure)
    assert result.message == "Could not open video: bad.mp4"
    assert result.is_recoverable is False
    db_mock.add_play_segment.assert_not_called()
    db_mock.add_event.assert_not_called()


def test_process_video_no_motion_returns_empty_success():
    """All-quiet motion (below threshold) yields a Success with no segments."""
    # 30 analysed frames all well below the 0.08 motion_threshold.
    cv2_mock, _ = _make_cv2_mock([0.0] * 30)
    db_mock = MagicMock()
    segmenter = _make_segmenter(db_mock=db_mock)

    with patch.object(motion_mod, "cv2", cv2_mock):
        result = segmenter.process_video("vid-1", "quiet.mp4")

    assert isinstance(result, Success)
    assert result.value == []
    db_mock.add_play_segment.assert_not_called()


def _high_motion_ratios():
    """A motion profile that produces exactly one segment long enough to survive
    the 3.0s min_segment_duration filter.

    fps=30, analysis_fps=5 -> frame_interval=6, so each analysed frame is 6 real
    frames apart -> 0.2s of video per analysed frame. We drive ~40 high-motion
    analysed frames (~8s) so the smoothed signal stays above threshold for a
    span well over 3s.
    """
    return [0.5] * 40


def test_process_video_fabricates_one_segment_deterministically():
    """With seeded ``random`` the fabricated segment + metadata are pinned exactly."""
    cv2_mock, _ = _make_cv2_mock(_high_motion_ratios())

    db_mock = MagicMock()
    # add_play_segment returns a deterministic id so the returned dict is pinned.
    db_mock.add_play_segment.return_value = 42

    segmenter = _make_segmenter(db_mock=db_mock)

    with patch.object(motion_mod, "cv2", cv2_mock):
        random.seed(0)
```

```
        result = segmenter.process_video("vid-1", "active.mp4")

    assert isinstance(result, Success)
    segments = result.value

    # === PIN: exactly one fabricated segment survives the min-duration filter ===
    assert len(segments) == 1
    seg = segments[0]

    # Pin the full returned dict shape/values (boundaries come from the motion
    # loop, server/receiver/metadata from seeded random).
    assert seg == _EXPECTED_SEGMENT

    # === PIN: the DB side effects ===
    db_mock.add_play_segment.assert_called_once()
    ps_args, ps_kwargs = db_mock.add_play_segment.call_args
    assert _EXPECTED_ADD_PLAY_SEGMENT_ARGS == (ps_args, ps_kwargs)

    # Events fabricated for the segment (serve + N smashes + termination).
    assert db_mock.add_event.call_count == _EXPECTED_EVENT_COUNT
    actual_events = [
        (call.args, call.kwargs) for call in db_mock.add_event.call_args_list
    ]
    assert actual_events == _EXPECTED_EVENTS
```

Placeholders filled in after a characterization run against UNCHANGED code.

```
_EXPECTED_SEGMENT = None
_EXPECTED_ADD_PLAY_SEGMENT_ARGS = None
_EXPECTED_EVENT_COUNT = None
_EXPECTED_EVENTS = None
```

19. user (2026-06-18T10:22:15.654Z)

```
File created successfully at: C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\wf_1d39e81c-851-1\tests\test_motion_segmenter.py (file state is
current in your context ? no need to Read it back)
```

20. assistant (2026-06-18T10:22:17.951Z)

```
Now let me run a characterization probe to capture the actual deterministic output values, so I
can fill in the expected constants. Let me write a temporary probe script.
```

21. user (2026-06-18T10:22:29.334Z)

```
TYPE Success
NSEG 1
--- SEGMENT ---
[{'duration': 7.999999999999999,
  'end_time': 8.166666666666666,
  'id': 42,
  'metadata': {'avg_speed_kmh': 51.7, 'intensity': 'medium', 'shot_count': 5},
  'receiver': 'Suresh',
  'server': 'Deepak',
  'start_time': 0.16666666666666666}]
--- add_play_segment.call_args ---
call('vid-1', 0.16666666666666666, 8.166666666666666, 'Deepak', 'Suresh', {'shot_count': 5,
'intensity': 'medium', 'avg_speed_kmh': 51.7})
--- add_event.count --- 4
--- events ---
((), {'segment_id': 42, 'timestamp': 0.6666666666666666, 'event_type': 'serve', 'player':
'Deepak', 'details': {'status': 'valid', 'type': 'high_serve'}})
((), {'segment_id': 42, 'timestamp': 6.99, 'event_type': 'smash', 'player': 'Suresh', 'details':
{'speed_kmh': 195.8}})
```

((), {'segment_id': 42, 'timestamp': 6.66, 'event_type': 'smash', 'player': 'Deepak', 'details': {'speed_kmh': 156.6}})
((), {'segment_id': 42, 'timestamp': 7.966666666666666, 'event_type': 'foul', 'player': 'Suresh', 'details': {'type': 'net_touch'}})

22. assistant (2026-06-18T10:22:31.838Z)

The output is deterministic. Now I have the exact pinned values. Let me fill in the expected constants in the test file. I'll use the exact float values captured.

23. user (2026-06-18T10:22:40.386Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\wf_1d39e81c-851-1\tests\test_motion_segmenter.py has been updated successfully. (file state is current in your context ? no need to Read it back)

24. assistant (2026-06-18T10:22:44.018Z)

The `_EXPECTED_SEGMENT` and `_EXPECTED_*` constants are defined at module scope after the test function that references them ? in Python that's fine since they're resolved at call time, not definition time. But for clarity let me verify the tests pass against the unchanged motion.py now.

25. user (2026-06-18T10:22:51.494Z)

```
============================= test session starts =============================
platform win32 -- Python 3.13.13, pytest-9.0.3, pluggy-1.6.0 --
C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\wf_1d39e81c-851-1
configfile: pytest.ini
plugins: anyio-4.13.0, cov-7.1.0
collecting ... collected 3 items

tests/test_motion_segmenter.py::test_process_video_failure_on_unopenable_video PASSED [ 33%]
tests/test_motion_segmenter.py::test_process_video_no_motion_returns_empty_success PASSED [ 66%]
tests/test_motion_segmenter.py::test_process_video_fabricates_one_segment_deterministically
PASSED [100%]

============================== 3 passed in 4.85s ==============================
```

26. assistant (2026-06-18T10:22:56.954Z)

All 3 characterization tests pass against the unchanged code. The behavior is pinned: Failure path, empty-Success path, and the fully-fabricated single-segment path with seeded random (server/receiver, metadata, 4 events in exact order).

Now let me perform the behavior-preserving refactor: extract the detection loop into `_detect_motion_segments` and the DB-populate half into `_populate_db_with_events`, with `process_video` as orchestrator. I must keep the random call order identical and all logic/log strings unchanged.

27. user (2026-06-18T10:23:04.723Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\wf_1d39e81c-851-1\backend\pipeline\segmenters\motion.py has been updated successfully. (file state is current in your context ? no need to Read it back)

28. assistant (2026-06-18T10:23:07.697Z)

Now I need to handle the transition between the detection loop (ending at the filter of `final_segments`) and the DB-populate half. The detection loop currently ends at line 157 with `final_segments = [...]`. I need to insert a `return final_segments` there, and start the `_populate_db_with_events` method. Let me view the current state around that boundary.

29. user (2026-06-18T10:23:08.862Z)

```
160                 curr_start, curr_end = raw_segments[0]
161                 for next_start, next_end in raw_segments[1:]:
162                     if next_start - curr_end < dead_time_merge_gap:
163                         # Merge
164                         curr_end = next_end
165                     else:
166                         merged_segments.append((curr_start, curr_end))
167                         curr_start, curr_end = next_start, next_end
168                 merged_segments.append((curr_start, curr_end))
169 
170             # Post-Processing: 2. Filter segments shorter than min_segment_duration
171             final_segments: List[Tuple[float, float]] = [
172                 r for r in merged_segments if (r[1] - r[0]) >= min_segment_duration
173             ]
174 
175             # Populate SQLite DB with segments and metadata events
176             segments_data = []
177             for start, end in final_segments:
178                 # Assign Server and Receiver randomly from pool
179                 server, receiver = random.sample(player_pool, 2)
180 
181                 # Formulate dynamic rich tags
182                 dur = end - start
183                 metadata = {
184                     "shot_count": random.randint(4, 25),
185                     "intensity": "high" if dur > 12 else "medium",
186                     "avg_speed_kmh": round(random.uniform(40, 85), 1)
187                 }
188 
189                 segment_id = self.db.add_play_segment(video_id, start, end, server,
receiver, metadata)
190 
191                 if segment_id:
192                     # Add sub-segment events
193                     # A: Serve Event (always at start)
194                     serve_foul = random.random() < 0.15 # 15% chance of serve foul
195                     self.db.add_event(
196                         segment_id=segment_id,
197                         timestamp=start + 0.5,
198                         event_type="serve",
199                         player=server,
200                         details={"status": "foul" if serve_foul else "valid", "type":
"low_serve" if random.random() > 0.5 else "high_serve"}
201                     )
202 
203                     # B: Smash Events (1 to 3 random smashes per segment)
204                     smashes_count = random.randint(0, 3)
205                     for _ in range(smashes_count):
206                         smash_time = random.uniform(start + 1.5, end - 1.0)
207                         smasher = random.choice([server, receiver])
208                         self.db.add_event(
209                             segment_id=segment_id,
210                             timestamp=round(smash_time, 2),
211                             event_type="smash",
212                             player=smasher,
213                             details={"speed_kmh": round(random.uniform(120, 250), 1)}
214                         )
215 
216                     # C: Segment Termination Event (foul/winner)
217                     foul_types = ["out_of_bounds", "net_touch", "double_hit", "fault"]
218                     is_foul = random.random() > 0.4
219                     error_player = random.choice([server, receiver])
220 
```

```
221                     self.db.add_event(
222                         segment_id=segment_id,
223                         timestamp=end - 0.2,
224                         event_type="foul" if is_foul else "winner",
225                         player=error_player,
226                         details={"type": random.choice(foul_types)} if is_foul else
{"shot_type": "drop" if random.random() > 0.5 else "smash"}
227                     )
228
229                     # Store segment representation to return to caller
230                     segments_data.append({
231                         "id": segment_id,
232                         "start_time": start,
233                         "end_time": end,
234                         "duration": dur,
235                         "server": server,
236                         "receiver": receiver,
237                         "metadata": metadata
238                     })
239
240             return Success(value=segments_data)
241
242     # Register the segmenter
243     segmenter_registry.register(MotionPlaySegmenter)
244
```

30. assistant (2026-06-18T10:23:14.972Z)

Now I'll replace the boundary: after `final_segments` is computed, return it from `_detect_motion_segments`, and convert the DB-populate half into the `_populate_db_with_events` method. The body of the loop stays byte-identical (same logic, same random call order, same return-shape).

31. user (2026-06-18T10:23:19.633Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\wf_1d39e81c-851-1\backend\pipeline\segmenters\motion.py has been updated successfully. (file state is current in your context ? no need to Read it back)

32. assistant (2026-06-18T10:23:22.137Z)

Now fix the final `return Success(value=segments_data)` ? that should now be `return segments_data` inside the `_populate_db_with_events` method (the `Success` wrapping moved up to `process_video`).

33. user (2026-06-18T10:23:25.211Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\wf_1d39e81c-851-1\backend\pipeline\segmenters\motion.py has been updated successfully. (file state is current in your context ? no need to Read it back)

34. assistant (2026-06-18T10:23:28.737Z)

Now let me read the full refactored file to verify correctness ? particularly that the `cap.release()` and the detection loop body remained intact and properly indented within `_detect_motion_segments`.

35. user (2026-06-18T10:23:29.553Z)

```
20          def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
21              """
22              Processes a video to detect play segments using motion heuristics,
23              populates SQLite with detected segments, and indexes detailed events.
```

```
24              """
25              if not player_pool:
26                  player_pool = ["Avi", "Suresh", "Ramesh", "Deepak"]
27
28              cap = cv2.VideoCapture(video_path)
29              if not cap.isOpened():
30                  logger.error(f"Segmenter could not open video: {video_path}")
31                  return Failure(message=f"Could not open video: {video_path}",
is_recoverable=False)
32
33              final_segments = self._detect_motion_segments(cap, max_frames)
34
35              segments_data = self._populate_db_with_events(video_id, final_segments,
player_pool)
36
37              return Success(value=segments_data)
38
39          def _detect_motion_segments(self, cap: Any, max_frames: int = None) ->
List[Tuple[float, float]]:
40              """Run the pixel-motion heuristic over ``cap`` and return the detected
41              (start, end) play segments.
42
43              Reads/sub-samples frames, builds a per-frame motion signal, smooths it,
44              thresholds it into raw segments, then merges dead-time gaps and filters
45              out segments shorter than ``min_segment_duration``. ``cap`` is released
46              before returning. This is the detection half of ``process_video`` ? it
47              performs no DB writes and no fabrication.
48              """
49              fps = cap.get(cv2.CAP_PROP_FPS)
50              total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
51              duration = total_frames / fps if fps > 0 else 0
52
53              # Sub-sample settings (5 FPS for analysis)
54              analysis_fps = 5.0
55              frame_interval = max(1, int(fps / analysis_fps))
56
57              idx_cfg = self.config.get("indexing", {})
58              motion_threshold = idx_cfg.get("motion_threshold", 0.08)
59              min_segment_duration = idx_cfg.get("min_segment_duration", 3.0)
60              dead_time_merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
61
62              prev_gray = None
63              motion_signals = []
64              timestamps = []
65
66              frame_idx = 0
67              processed_count = 0
68              t_start = time.time()
69              # Emit a progress line roughly every 5% of the video so long runs are
70              # observable instead of silent (a full 1080p match is ~tens of minutes).
71              progress_step = max(1, total_frames // 20) if total_frames > 0 else 0
72              next_progress = progress_step
73              logger.info(f"[motion] start: {total_frames} frames @ {fps:.1f}fps "
74                          f"(~{duration/60:.1f} min), analyzing every {frame_interval}th
frame")
75              while True:
76                  # Skip decoding for intermediate frames using cap.grab() for high
performance
77                  for _ in range(frame_interval - 1):
78                      if not cap.grab():
79                          break
```

36. assistant (2026-06-18T10:23:34.092Z)

The refactor looks structurally correct. The detection half (lines 39-173) reads config, runs

the loop, releases cap, and returns `final_segments`. The DB-populate half is a separate method.
Now let me run the characterization tests again to confirm they still pass unchanged.

37. user (2026-06-18T10:23:42.676Z)

```
============================= test session starts =============================
platform win32 -- Python 3.13.13, pytest-9.0.3, pluggy-1.6.0 --
C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\wf_1d39e81c-851-1
configfile: pytest.ini
plugins: anyio-4.13.0, cov-7.1.0
collecting ... collected 3 items

tests/test_motion_segmenter.py::test_process_video_failure_on_unopenable_video PASSED [ 33%]
tests/test_motion_segmenter.py::test_process_video_no_motion_returns_empty_success PASSED [ 66%]
tests/test_motion_segmenter.py::test_process_video_fabricates_one_segment_deterministically
PASSED [100%]

============================== 3 passed in 4.96s ==============================
```

38. assistant (2026-06-18T10:23:45.914Z)

The same characterization tests pass unchanged after the refactor. The behavior is preserved.
Let me check the `Tuple` import is present (it's used in `_detect_motion_segments` and `_populate_db_with_events` signatures) ? line 4 has `from typing import List, Dict, Any, Tuple`. Good.

Now let me run the full gate. First the offline suite.

39. user (2026-06-18T10:24:55.366Z)

```
================================================================================
 Full test suite:  C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe -m pytest
-q
 obsreport (optional reporting dep): available
================================================================================
.....................ss.................................................. [  8%]
........................................................................... [ 16%]
........................................................................... [ 24%]
........................................................................... [ 33%]
........................................................................... [ 41%]
........................................................................... [ 49%]
........................................................................... [ 58%]
........................................................................... [ 66%]
........................................................................... [ 74%]
...........sssss............................................................ [ 83%]
........................................................................... [ 91%]
........................................................................... [ 99%]
...                                                                        [100%]
860 passed, 7 skipped in 65.17s (0:01:05)
================================================================================
 RESULT: PASS - all tests green
================================================================================
```

40. assistant (2026-06-18T10:24:57.597Z)

Full suite green: 860 passed, 7 skipped. Now ruff and mypy.

41. user (2026-06-18T10:25:37.583Z)

```
All checks passed!
===RUFF_EXIT=0===
backend\annotations\base.py:85: note: By default the bodies of untyped functions are not
```

```
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\sports\__init__.py:7: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs  [annotation-unchecked]
backend\pipeline\validators\base.py:36: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 107 source files
===MYPY_EXIT=0===
```

42. assistant (2026-06-18T10:25:42.156Z)

Ruff clean, mypy clean (only informational notes, "Success: no issues found"). Now the coverage gate. The motion.py file is omitted in `.coveragerc`, so this refactor doesn't directly change its coverage line, but I must confirm overall coverage stays at/above the floor and doesn't regress vs base (~80.9%).

43. user (2026-06-18T10:27:26.024Z)

```
backend\tools\__init__.py                              0      0   100%
backend\tools\export.py                               75      2    97%
backend\tools\rally_slicer.py                        104     47    55%
backend\utils\__init__.py                              0      0   100%
backend\utils\geometry.py                             29      0   100%
backend\utils\hashing.py                               9      0   100%
backend\utils\logging_setup.py                        11      0   100%
backend\utils\paths.py                                27      3    89%
backend\utils\run_telemetry.py                       126     12    90%
backend\utils\video.py                                25      3    88%
training\__init__.py                                   0      0   100%
training\gen0\__init__.py                              0      0   100%
training\gen0\harness.py                             108     11    90%
----------------------------------------------------------------------
TOTAL                                               8092   1542    81%
Required test coverage of 70% reached. Total coverage: 80.94%
860 passed, 7 skipped, 7 warnings in 98.73s (0:01:38)
======================================================================
 RESULT: PASS - all tests green
======================================================================
```

44. assistant (2026-06-18T10:27:30.806Z)

All four gates green:
- Suite: 860 passed, 7 skipped
- ruff: All checks passed
- mypy: Success, no issues
- Coverage: 80.94%, matches base ~80.9%, well above the 70% floor (no regression)

The motion.py file remains omitted in `.coveragerc` (unchanged), so coverage accounting is unaffected by the new methods. The new test file adds 3 tests that pin the behavior.

Let me verify my diff is exactly the intended files and commit. Let me check the git status and diff first.

45. user (2026-06-18T10:27:33.322Z)

```
 M backend/pipeline/segmenters/motion.py
?? tests/test_motion_segmenter.py
===DIFFSTAT===
 backend/pipeline/segmenters/motion.py | 30 ++++++++++++++++++++++++++++++-
 1 file changed, 29 insertions(+), 1 deletion(-)
===BRANCH===
refactor/motion-process-video-tests-then-helpers
===HEAD===
```

e67add1 refactor(api): extract chunked-upload island from main into backend.api.uploads router
(#228)
===ONLY MINE vs origin/master===

46. assistant (2026-06-18T10:27:36.631Z)

The working tree has exactly the two intended files: `backend/pipeline/segmenters/motion.py`
(modified) and `tests/test_motion_segmenter.py` (new). HEAD is still origin/master (no sibling
commits crept in ? `origin/master..HEAD` is empty, confirming a clean base). Let me do a final
review of the complete refactored motion.py to ensure logic is byte-for-byte preserved, then
commit.

47. user (2026-06-18T10:27:37.720Z)

```
diff --git a/backend/pipeline/segmenters/motion.py b/backend/pipeline/segmenters/motion.py
index 545b47a..bf8274c 100644
--- a/backend/pipeline/segmenters/motion.py
+++ b/backend/pipeline/segmenters/motion.py
@@ -30,6 +30,22 @@ class MotionPlaySegmenter(BasePlaySegmenter):
             logger.error(f"Segmenter could not open video: {video_path}")
             return Failure(message=f"Could not open video: {video_path}", is_recoverable=False)

+        final_segments = self._detect_motion_segments(cap, max_frames)
+
+        segments_data = self._populate_db_with_events(video_id, final_segments, player_pool)
+
+        return Success(value=segments_data)
+
+    def _detect_motion_segments(self, cap: Any, max_frames: int = None) -> List[Tuple[float,
float]]:
+        """Run the pixel-motion heuristic over ``cap`` and return the detected
+        (start, end) play segments.
+
+        Reads/sub-samples frames, builds a per-frame motion signal, smooths it,
+        thresholds it into raw segments, then merges dead-time gaps and filters
+        out segments shorter than ``min_segment_duration``. ``cap`` is released
+        before returning. This is the detection half of ``process_video`` ? it
+        performs no DB writes and no fabrication.
+        """
         fps = cap.get(cv2.CAP_PROP_FPS)
         total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
         duration = total_frames / fps if fps > 0 else 0
@@ -156,6 +172,18 @@ class MotionPlaySegmenter(BasePlaySegmenter):
             r for r in merged_segments if (r[1] - r[0]) >= min_segment_duration
         ]

+        return final_segments
+
+    def _populate_db_with_events(self, video_id: str, final_segments: List[Tuple[float,
float]],
+                                 player_pool: List[str]) -> List[Dict[str, Any]]:
+        """Persist ``final_segments`` and their fabricated metadata/events to the DB.
+
+        For each detected segment this assigns a random server/receiver, fabricates
+        rich tags, then indexes a serve / 0?3 smash / termination event set ? all
+        with the stdlib ``random`` module (the legacy demo's fabricated baseline).
+        Returns the per-segment dicts handed back to the caller. The ``random``
+        call order here is load-bearing for reproducibility and must not change.
+        """
         # Populate SQLite DB with segments and metadata events
         segments_data = []
         for start, end in final_segments:
@@ -221,7 +249,7 @@ class MotionPlaySegmenter(BasePlaySegmenter):
                     "metadata": metadata
                 })
```

```
-        return Success(value=segments_data)
+        return segments_data

 # Register the segmenter
 segmenter_registry.register(MotionPlaySegmenter)
```

48. assistant (2026-06-18T10:27:42.452Z)

The diff is a clean, pure structural extraction:
- `process_video` now orchestrates: calls `_detect_motion_segments(cap, max_frames)`, then `_populate_db_with_events(...)`, wraps in `Success`.
- The detection loop body and DB-populate loop body are moved verbatim (no logic, threshold, log-string, or random-call-order change).
- The only real edit inside the moved bodies: `return Success(value=segments_data)` ? `return segments_data` (the `Success` wrap moved up to the orchestrator). All thresholds, constants, log levels/strings, and ordering are identical.

Now let me commit with explicit paths.

49. user (2026-06-18T10:27:44.903Z)

```
warning: in the working copy of 'tests/test_motion_segmenter.py', LF will be replaced by CRLF
the next time Git touches it
M  backend/pipeline/segmenters/motion.py
A  tests/test_motion_segmenter.py
```

50. assistant (2026-06-18T10:27:47.965Z)

Both files staged (the LF/CRLF warning is benign, just git's autocrlf normalization). Let me re-verify HEAD one more time right before commit (concurrency safety) and commit.

51. user (2026-06-18T10:27:56.455Z)

```
===still empty means clean base===
[refactor/motion-process-video-tests-then-helpers 3057ada] refactor(motion): extract detection +
DB-populate halves from process_video
 2 files changed, 251 insertions(+), 1 deletion(-)
 create mode 100644 tests/test_motion_segmenter.py
===COMMITTED===
3057ada refactor(motion): extract detection + DB-populate halves from process_video
```

52. assistant (2026-06-18T10:28:00.913Z)

Committed as `3057ada`. Now push and open the PR.

53. user (2026-06-18T10:28:04.493Z)

```
remote:
remote: Create a pull request for 'refactor/motion-process-video-tests-then-helpers' on GitHub
by visiting:
remote:      https://github.com/avidullu/badminton-highlight-indexer/pull/new/refactor/motion-
process-video-tests-then-helpers
remote:
branch 'refactor/motion-process-video-tests-then-helpers' set up to track
'origin/refactor/motion-process-video-tests-then-helpers'.
To https://github.com/avidullu/badminton-highlight-indexer.git
 * [new branch]      refactor/motion-process-video-tests-then-helpers -> refactor/motion-
process-video-tests-then-helpers
```

54. assistant (2026-06-18T10:28:06.348Z)

Pushed. Now create the PR.

55. user (2026-06-18T10:28:22.473Z)

https://github.com/avidullu/badminton-highlight-indexer/pull/230

56. assistant (2026-06-18T10:28:26.507Z)

PR #230 opened. All four gates passed before the push, the branch is based cleanly on origin/master, and only my two intended files changed. Let me capture the final diffstat for the structured report.

57. user (2026-06-18T10:28:28.686Z)

```
    fabricated single-segment + events. They pass identically before and after
    the extraction.

    Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>

 backend/pipeline/segmenters/motion.py |  30 ++++-
 tests/test_motion_segmenter.py        | 222 ++++++++++++++++++++++++++++++++++
 2 files changed, 251 insertions(+), 1 deletion(-)
```

58. assistant (2026-06-18T10:28:31.230Z)

All work complete. Let me return the structured result.

59. user (2026-06-18T10:28:52.619Z)

Structured output provided successfully

60. assistant (2026-06-18T10:29:02.424Z)

Done. PR #230 is open.

Summary of what I did, following the characterization-tests-first discipline:

1. **Setup** ? branched `refactor/motion-process-video-tests-then-helpers` from `origin/master` (verified HEAD == origin/master, and `origin/master..HEAD` listed only my commit before pushing).

2. **Characterization tests first** ? wrote `tests/test_motion_segmenter.py` (3 tests) that mock the module-level `cv2` (the whole module, since `process_video` calls cv2 transforms directly) and seed `random`, pinning the Failure path, the empty-Success path, and the exact fabricated single-segment + 4 events. Confirmed all 3 **pass against the unchanged `motion.py`**.

3. **Refactor** ? extracted `_detect_motion_segments` (detection loop) and `_populate_db_with_events` (DB-populate/fabrication half); `process_video` is now a thin orchestrator returning the same `Success`/`Failure`. Pure move ? the only in-body edit was hoisting the `Success(...)` wrap up to the orchestrator.

4. **Re-verify** ? the same characterization tests pass unchanged.

Gate (all four green, run locally with cv2/torch/numpy present):
- Suite: 860 passed, 7 skipped
- ruff: All checks passed
- mypy: Success, no issues (107 files)
- Coverage: 80.94% ? matches base ~80.9%, no regression, floor 70%. `motion.py` stays omitted in `.coveragerc` (unchanged).

Relevant files (absolute paths in this worktree):
- `C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\wf_1d39e81c-851-1\backend\pipeline\segmenters\motion.py`
- `C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\wf_1d39e81c-851-1\tests\test_motion_segmenter.py`

PR: https://github.com/avidullu/badminton-highlight-indexer/pull/230 (no merge, as instructed).